통합LIMS WEB API 검사기관 개발가이드

2020.08

식 품 의 약 품 안 전 처

< 목 차 >

1. 개요 ····· 4
1.1. 문서개요 ····· 4
1.1.1. 목적 ····· 4
1.1.2. 범위 ····· 4
2. WEB API 개발 가이드 ····· 4
2.1. 개요 ····· 4
2.2. Software Architecture ····· 4
2.3. 인증 및 권한처리 ····· 4
2.4. Service Architecture ····· 5
2.5. 공통 개발가이드 ····· 5
2.5.1. 사전준비사항 ····· 5
2.5.2. 인증서정보 ····· 6
2.5.3. WEB API WSDL &WADL URL ····· 6
2.5.4. 입력 및 출력 형식 ····· 6
2.5.5. 결과반환 ····· 7
2.6. SOAP 개발가이드 ····· 7
2.6.1. 개발환경설정 ····· 7
2.6.2. 서비스호출방법 ····· 8
2.7. RESTful 개발가이드 ····· 10
2.7.1. 서비스호출 절차 ····· 10
2.8. UI 구성예제 ····· 12
2.9. 통합LIMS &검사기관 시스템 양방향 통신 ····· 13
2.9.1. 사전준비사항 ····· 13
2.9.2. 처리순서 ····· 13
2.9.3. 통합LIMS 주요 변경 이벤트 ····· 14
3. 중간 모듈(Web Service) ····· 15
3.1. 개요 ····· 15
3.2. 중간 모듈 프로세스 ····· 15
3.3. 중간 모듈 가이드 ····· 16
3.3.1. 서비스 환경설정 ····· 16
3.3.2. 서비스 운영방법 ····· 16

1. 개요

1.1. 문서개요

1.1.1. 목적

- 본 문서는 검사기관이 통합LIMS WEB API를 통해 식품의약품안전처의 통합LIMS 기능을 호출하여 시스템을 수정/개발할 경우에 필요한 정보를 제공하는데에 그 목적이 있다.

1.1.2. 범위

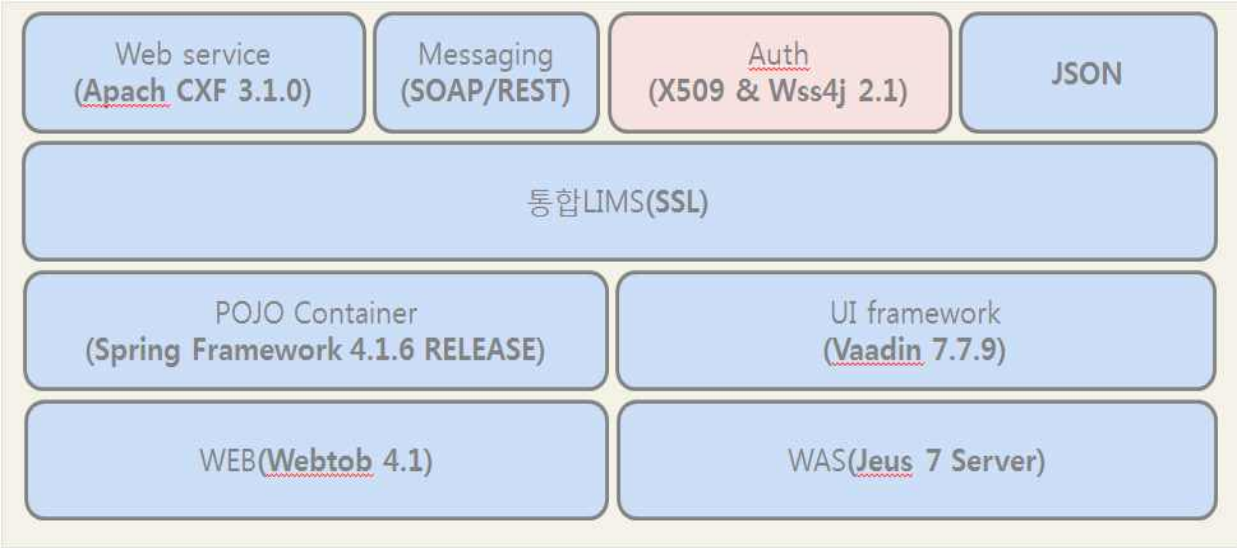
- 본 문서의 범위는 통합LIMS WEB API 서비스에 대한 내용으로 한정된다.

2. WEB API 개발 가이드

2.1. 개요

- 통합LIMS WEB API는 기존 통합LIMS 시스템의 기능을 Websevice형태로 만들어 자체시스템을 보유 중 이거나 신규로 구축예정인 검사기관이 해당 Webservice를 활용하여 기관의 검사데이터를 통합LIMS로 실시간 전송할 수 있게 한다.

2.2. Software Architecture

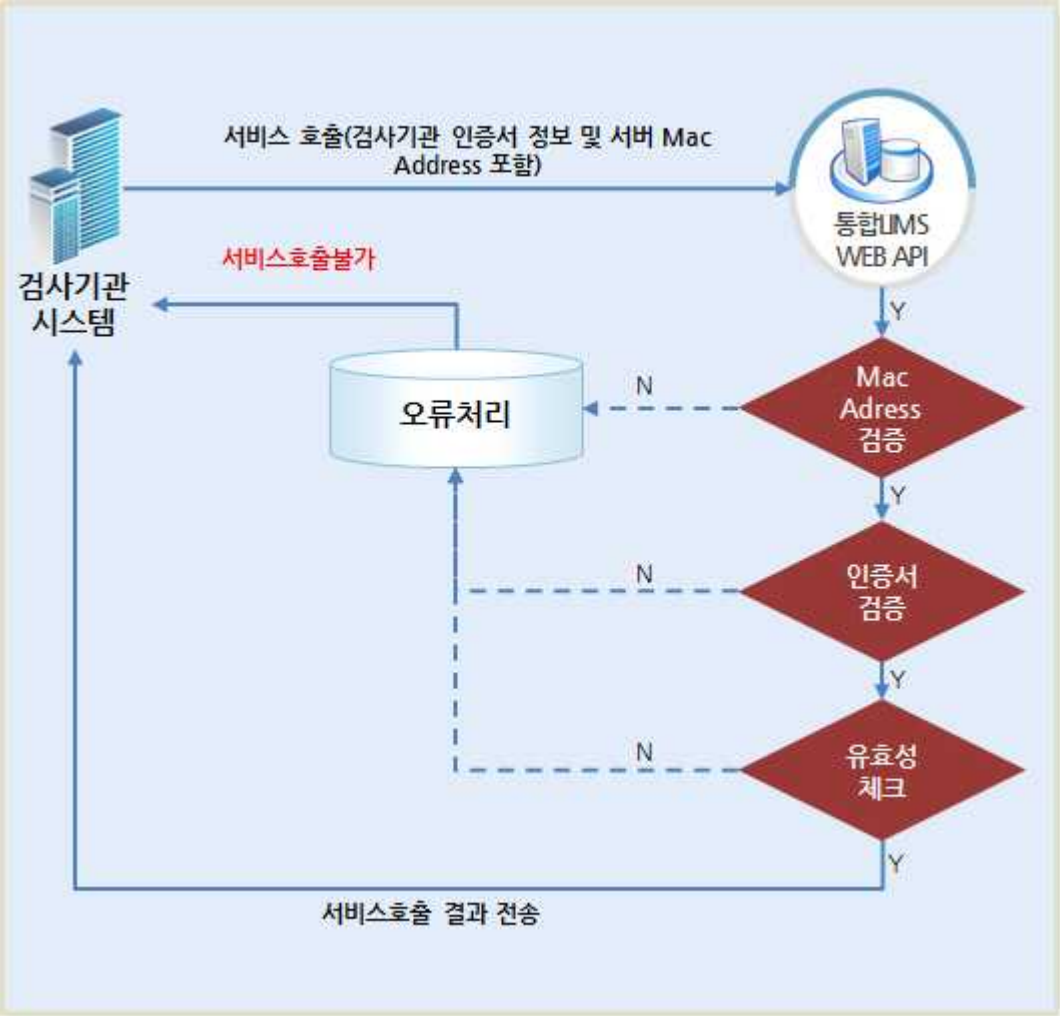


- 통합LIMS WEB API는 기존 통합LIMS 시스템기반위에 구축되어 있으며 SSL 과 Apache CXF, X509 인증으로 보안관련 부분을 처리하였다. 또한, SOAP과 REST방식의 두가지 방식의 웹서비스를 모두 제공하기 위해 입력과 출력 모두 JSON으로 처리하였다.

2.3. 인증 및 권한처리

로그인 인증	x509(version 3) 인증서를 이용한 PKI(공개키기반) 인증방식
서비스 호출 시 인증	클라이언트 서버의 Mac Address 인증을 이용하여 지정된 서버에서만 웹서비스 호출 가능 하도록 처리
권한처리	검사기관 사용자 별 통합LIMS의 권한을 승계하여 각 웹 서비스 모듈 호출 시 권한체크

2.4. Service Architecture



1) 검사기관이 클라이언트 인증서정보를 탑재하여 통합LIMS WEB API 서비스호출
2) 통합LIMS WEB API 서버에서 서버인증서와 검사기관 인증서를 비교하여 유효성 확인
3) 검사기관의 서버 Mac Address 유효성 확인
4) 인증서 및 Mac Address 유효성 통과 시 요청 웹서비스 호출
5) 요청 웹서비스의 입력파라미터값을 검증 후 Validation Check
6) 처리결과 리턴

2.5. 공통 개발가이드

2.5.1. 사전준비사항

1) 식품의약품안전처에 통합LIMS WEB API 사용 신청
   - 식약처 정보화도움터(043-234-3100) 또는 정보화담당관실(043-719-1611)로 문의
2) 식약처에서 사용승인 후 [시스템관리]-[시스템관리]-[검사기관관리] 화면에서 API 인증서 다운로드
3) 검사기관 시스템에 인증서 설정 후 API 서비스 호출

※ WEB API 신청방식은 통합LIMS내 신청화면 제공 예정

2.5.2. 인증서정보

최상위기관	식품의약품안전처
버전	V3
서명알고리즘	SHA256RSA
서명 해시 알고리즘	SHA256
공개키	RSA(2048 Bits)
인증서타입	JKS

2.5.3. WEB API WSDL &WADL URL

- https://wslims.mfds.go.kr/webService

2.5.4. 입력 및 출력 형식

- 입력 및 출력형식은 SOAP 및 REST 방식 모두 JSON 을 이용함.

1) 입력파라미터

가) 파라미터구조 : 파라미터명은 camel 표기법을 따름 (ex : inspctInsttCode)

나) 파라미터 타입

타입	형식	사용예
A Type	Key &Value	{"A_key" : "A_value", "B_key" : "B_value"}
B Type	Key &Object	{"A_key" : "A_value", "B_key" : {"a_key" : "a_value", "b_key" : "b_value"}}
C Type	Key &List	{"A_key" : "A_value", "B_key" : [{"a_key" : "a_value", "b_key" : "b_value"}, {"a_key" : "a_value", "b_key" : "b_value"}]}

2) 파라미터입력시 주의사항

가) 입력 **Parameter**중 공통 코드 값은 공통코드목록 집에 있는 코드 외에는 입력불가

나) 입력 **Parameter**중 mfdsLimsId, psitnInsttCode는 사용자 인증에 사용되므로 반드시 필수로 포함

- 올바른 예 : {"mfdsLimsId" : "tester", "psitnInsttCode" : "0000000","A_key" : "A_value" "B_key" : "B_value"}
- 잘못된 예 : (mfdsLimsId, psitnInsttCode가 하위 Object에 있을 때)
  {"A_key" : "A_value", "B_key" : [{"mfdsLimsId" : "tester","psitnInsttCode":"00000"}]}

3) Validation Check

- 통합LIMS의 Validation Check와 동일한 패턴을 가짐
- 조회 및 입력 시 필요한 Parameter가입력되지 않거나 올바르지 않은 값이 입력되면 Validation Check Error 발생

ex) 성공 : {"resultFlag" : "1", "resultData":[{"a_key" : "a_value", "b_key" : "b_value"}, {"a_key" : "a_value", "b_key" : "b_value"}] }

실패 : {"resultFlag":"0","validate":"false","errorMessage":"사용자 정보가 존재하지 않습니다."}

※ 서비스 별 필요한 입력 Parameter정보는 서비스 별 API 가이드 참조

2.5.5. 결과반환

명칭	내용	예시	비고
resultFlag	요청 성공 여부	1:성공, 0:실패	
validate	validation실패 여부	validate : false	
errorMessage	호출 실패 시 에러 메시지	errorMessage : 사용자의 기관정보가 존재하지 않습니다.	
apiError	시스템 오류 시 발생	{"apiError":"null 자세한 내용은 WEB API 관리자에게 문의하세요.","resultFlag":"0"}	
header	요청한 데이터의 header	{"inspctDeptCode":"부서코드","userName":"사용자명","sploreReqestNo":"의뢰번호"}	서비스 별 Return 내용은 별도 배포되는 서비스 별 API 가이드 참조
resultData	요청 성공 시 데이터 결과값	{"resultData":[{"a_key" : "a_value", "b_key" : "b_value"}, {"a_key" : "a_value", "b_key" : "b_value"}], "resultFlag" : "1"}	

2.6. 개발가이드

- 본 문서에서는 eclipse + java + spring 기반의 개발환경을 기본으로 설명한다.

2.6.1. 개발환경설정

1) applicationContext.xml 설정

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context"
        xmlns:jaxws="http://cxf.apache.org/jaxws"
        xmlns:jaxrs="http://cxf.apache.org/jaxrs"
        xmlns:cxf="http://cxf.apache.org/core"
        xmlns:sec="http://cxf.apache.org/configuration/security"
        xmlns:http="http://cxf.apache.org/transports/http/configuration"
        xmlns:httpj="http://cxf.apache.org/transports/http-jetty/configuration"
        xmlns:util="http://www.springframework.org/schema/util"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
                            http://www.springframework.org/schema/beans/spring-beans.xsd
                            http://www.springframework.org/schema/context
                            http://www.springframework.org/schema/context/spring-context.xsd
                            http://cxf.apache.org/bindings/soap
                            http://cxf.apache.org/schemas/configuration/soap.xsd
                            http://cxf.apache.org/core
                            http://cxf.apache.org/schemas/core.xsd
                            http://cxf.apache.org/jaxws
                            http://cxf.apache.org/schemas/jaxws.xsd
                            http://cxf.apache.org/jaxrs
                            http://cxf.apache.org/schemas/jaxrs.xsd
                            http://cxf.apache.org/configuration/security
                            http://cxf.apache.org/schemas/configuration/security.xsd
                            http://cxf.apache.org/transports/http/configuration
                            http://cxf.apache.org/schemas/configuration/http-conf.xsd
                            http://cxf.apache.org/transports/http-jetty/configuration
                            http://cxf.apache.org/schemas/configuration/http-jetty.xsd
                            http://www.springframework.org/schema/util
                            http://www.springframework.org/schema/util/spring-util-4.0.xsd">

    <util:properties id="config" location="classpath:application.properties"/>

    <import resource="classpath:META-INF/cxf/cxf.xml"/>
    <import resource="classpath:META-INF/cxf/cxf-servlet.xml"/>

    <bean id="logInBound" class="org.apache.cxf.interceptor.LoggingInInterceptor" />
    <bean id="logOutBound" class="org.apache.cxf.interceptor.LoggingOutInterceptor" />

    <jaxws:client id="LMWWA01F010Service"
        serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F010Service"
        address="#{config['wslimsUrl']}/webService/LMWWA01F010">
            <jaxws:inInterceptors>
                    <ref bean="logInBound" />
            </jaxws:inInterceptors>
            <jaxws:outInterceptors>
                    <ref bean="logOutBound" />
            </jaxws:outInterceptors>
    </jaxws:client>
    <jaxws:client id="LMWWA01F020Service"
        serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F020Service"
        address="#{config['wslimsUrl']}/webService/LMWWA01F020">
            <jaxws:inInterceptors>
```

```
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>
        <jaxws:client id="LMWWA01F030Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F030Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F030">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>
        <jaxws:client id="LMWWA01F040Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F040Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F040">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>
        <jaxws:client id="LMWWA01F050Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F050Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F050">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>
        <jaxws:client id="LMWWA01F060Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F060Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F060">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>

        <jaxws:client id="LMWWA01F070Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F070Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F070">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>
        <jaxws:client id="LMWWA01F080Service"
            serviceClass="gov.mfds.lms.server.lm.w.wa.wwa01.service.LMWWA01F080Service"
            address="#{config['wslimsUrl']}/webService/LMWWA01F080">
                    <jaxws:inInterceptors>
                        <ref bean="logInBound" />
                    </jaxws:inInterceptors>
                    <jaxws:outInterceptors>
                        <ref bean="logOutBound" />
                    </jaxws:outInterceptors>
        </jaxws:client>

        <!-- WSS4JOutInterceptor for signing and encrypting outbound SOAP -->
    <bean class="org.apache.cxf.ws.security.wss4j.WSS4JOutInterceptor" id="outbound-security">
        <constructor-arg>
            <map>
                <entry key="action" value="Timestamp Signature Encrypt"/>
                <entry key="user" value="#{config['keyStore.keyAlias']}"/>
                <entry key="signatureKeyIdentifier" value="DirectReference"/>
                <entry key="encryptionUser" value="1"/>
                <entry key="signatureParts"
                  value="{Element}{http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility
                  -1.0.xsd}Timestamp;{Element}{http://schemas.xmlsoap.org/soap/envelope/}Body"/>
                <entry key="encryptionParts"
                  value="{Element}{http://www.w3.org/2000/09/xmldsig#}Signature;{Content}{http://schemas.xml
                  soap.org/soap/envelope/}Body"/>
                <entry key="encryptionSymAlgorithm" value="http://www.w3.org/2001/04/xmlenc#tripledes-cbc"/>
            </map>
        </constructor-arg>
    </bean>

    <!--  WSS4JInInterceptor for decrypting and validating the signature of inbound SOAP -->
    <bean class="org.apache.cxf.ws.security.wss4j.WSS4JInInterceptor" id="inbound-security">
        <constructor-arg>
            <map>
                <entry key="action" value="Timestamp Signature Encrypt"/>
```

```
            </map>
        </constructor-arg>
    </bean>

        <bean id="restWebApiService" class="gov.mfds.lms_client.WebApiService"></bean>

        <jaxrs:server id="restWebservice" address="/rest">
                <jaxrs:serviceBeans>
                        <ref bean="restWebApiService"/>
                </jaxrs:serviceBeans>
        </jaxrs:server>

</beans>
```

2) servlet-context.xml 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:mvc="http://www.springframework.org/schema/mvc"
        xmlns:context="http://www.springframework.org/schema/context"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
                http://www.springframework.org/schema/beans
                http://www.springframework.org/schema/beans/spring-beans.xsd
                http://www.springframework.org/schema/mvc
                http://www.springframework.org/schema/mvc/spring-mvc.xsd
                http://www.springframework.org/schema/context
                http://www.springframework.org/schema/context/spring-context.xsd">

        <mvc:annotation-driven/>

        <context:component-scan base-package="gov.mfds.lms_client.Controller"/>
        <mvc:resources mapping="/resources/**" location="/resources/"/>

        <bean id="multipartResolver"
                class="org.springframework.web.multipart.commons.CommonsMultipartResolver">
                <property name="maxUploadSize" value="52428800"></property>
                <property name="maxInMemorySize" value="52428800"></property>
        </bean>

        <bean id="viewResolver"
                class="org.springframework.web.servlet.view.UrlBasedViewResolver">
                <property name="viewClass" value="org.springframework.web.servlet.view.JstlView"/>
                <property name="prefix" value="/WEB-INF/view/"/>
                <property name="suffix" value=".jsp"/>
        </bean>
</beans>
```

3) web.xml 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                xmlns="http://java.sun.com/xml/ns/javaee"
                xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
                        http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" version="3.0">

        <display-name>lms_client</display-name>
        <description>lms_client_project</description>

        <listener>
                <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
        </listener>
        <context-param>
                <param-name>contextConfigLocation</param-name>
                <param-value>
                        /WEB-INF/spring/applicationContext.xml
                </param-value>
        </context-param>

        <servlet>
                <servlet-name>dispatcher</servlet-name>
                <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
                <init-param>
                        <param-name>contextConfigLocation</param-name>
                        <param-value>/WEB-INF/spring/appServlet/servlet-context.xml</param-value>
                </init-param>
                <load-on-startup>1</load-on-startup>
        </servlet>

        <servlet-mapping>
                <servlet-name>dispatcher</servlet-name>
                <url-pattern>/</url-pattern>
        </servlet-mapping>

        <filter>
                <filter-name>encodingFilter</filter-name>
                <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
                <init-param>
```

```
                                <param-name>encoding</param-name>
                                <param-value>UTF-8</param-value>
                        </init-param>
            </filter>

            <filter-mapping>
                        <filter-name>encodingFilter</filter-name>
                        <url-pattern>/*</url-pattern>
            </filter-mapping>

            <session-config>
            <session-timeout>30</session-timeout>
            </session-config>


            <!-- CXFServlet 설정 -->
            <servlet>
                        <servlet-name>CXFServlet</servlet-name>
                        <servlet-class>org.apache.cxf.transport.servlet.CXFServlet</servlet-class>
                        <load-on-startup>1</load-on-startup>
            </servlet>
            <servlet-mapping>
                        <servlet-name>CXFServlet</servlet-name>
                        <url-pattern>/webService/*</url-pattern>
            </servlet-mapping>

</web-app>
```

4) application.properties 파일

```
keyStore.keyStoreLoc=C:/CA/O000170.jks
KeyStore.keyPassWord=mfds2015
keyStore.keyAlias=O000170
fileDownloadPath=C:/mfdshome/files/client
wslimsUrl=https://wslims.mfds.go.kr
```

2.6.2. 서비스호출방법

1) 파라미터 설정 : HashMap에 담아 JsonObject형태로 변환한다.

```
/* JSON PARMATER */
HashMap<String, String> pValue = new HashMap<String, String>();
pValue.put("psitnInsttCode", "1471000");
pValue.put("mfdsLimsId", "testId");
pValue.put("sploreReqestNo", "T0180400001");

JSONObject jsonobject = new JSONObject();
jsonobject.putAll(pValue);
```

2) 인증정보 설정

```
private ApplicationContext context = ContextLoader.getCurrentWebApplicationContext();

final private static ResourceBundle resource = ResourceBundle.getBundle("application");

//인증에 사용되는 jks 파일 경로
private String keyStoreLoc = resource.getString("keyStore.keyStoreLoc");

//인증에 사용되는 구분자
private String alias = resource.getString("keyStore.keyAlias");

//인증에 사용되는 키
private String passWord = resource.getString("KeyStore.keyPassWord");

//파일을 저장할 서버의 경로
private String localStorage = resource.getString("fileDownloadPath");

//해당 기관의 URL
private String wslimsUrl = resource.getString("wslimsUrl");

AuthenticateCert ac = new AuthenticateCert(keyStoreLoc, alias, passWord, context);
```

3) 조회서비스 호출 전체 소스

```
package gov.mfds.lms_client.Controller;

import java.util.HashMap;
import java.util.ResourceBundle;
import javax.servlet.http.HttpServletRequest;
import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.client.HttpClient;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.util.EntityUtils;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.springframework.context.ApplicationContext;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.context.ContextLoader;
import org.springframework.web.multipart.MultipartHttpServletRequest;
import com.authSecure.AuthenticateCert;


@Controller
public class WebApiController {

		private ApplicationContext context = ContextLoader.getCurrentWebApplicationContext();

		final private static ResourceBundle resource = ResourceBundle.getBundle("application");

		//인증에 사용되는 jks 파일 경로
		private String keyStoreLoc = resource.getString("keyStore.keyStoreLoc");

		//인증에 사용되는 구분자
		private String alias = resource.getString("keyStore.keyAlias");

		//인증에 사용되는 키
		private String passWord = resource.getString("KeyStore.keyPassWord");

		//파일을 저장할 서버의 경로
		private String localStorage = resource.getString("fileDownloadPath");

		//해당 기관의 URL
		private String wslimsUrl = resource.getString("wslimsUrl");

		AuthenticateCert ac = new AuthenticateCert(keyStoreLoc, alias, passWord, context);

		/**
		 * 통합 LIMS WEB API 테스트 호출
		 * @param model
		 * @return
		 */
		@RequestMapping(value = "/", method = {RequestMethod.GET, RequestMethod.POST})
		public String unitTest(Model model){
			return "unitTest";
		}

		/**
		 * 시나리오 상세정보 화면으로 이동
		 * @param model
		 * @return
		 */
		@RequestMapping(value = "/unitTestDetail", method = {RequestMethod.GET, RequestMethod.POST})
		public String unitTestDetail(Model model){
			return "unitTestDetail";
		}

		/**
		 * 파라미터 만들기 화면으로 이동
		 * @param model
		 * @return
		 */
		@RequestMapping(value = "/createParameters", method = {RequestMethod.GET,
			RequestMethod.POST})
		public String createParameters(Model model, HttpServletRequest request){
			return "createParameters";
		}

		/**
		 * 파일 다운로드 호출
		 * @param model
		 * @return
		 */
		@RequestMapping(value = "/unitTestFileDownLoad", method = {RequestMethod.GET,
			RequestMethod.POST})
		public String unitTestFileDownLoad(Model model){
			return "unitTestFileDownLoad";
		}
```

```java
	/**
	 * 공통 Rest 호출
	 * @param url
	 * @param param
	 * @return
	 */
	public String callWebApiRest(String url, String param){
		return ac.callWebApiRest(url, param, localStorage);
	}

	/**
	 * 공통 Soap 호출
	 * @param url
	 * @param param
	 * @return
	 */
	public String callWebApiSoap(String url, String param){
		return ac.callWebApiSoap(url, param, localStorage);
	}

	/**
	 * Rest, Soap 호출을 위한 함수
	 * @param param
	 * @return
	 */
@RequestMapping(value = "/selectUnitTest", method = {RequestMethod.GET, RequestMethod.POST})
	public @ResponseBody String selectUnitTest(@RequestBody String param){

		String result = "";
		String map = "";

		try{
			JSONParser parser = new JSONParser();
			JSONObject jsonResult = (JSONObject)parser.parse(param);

			map = String.valueOf(jsonResult.get("param"));

			//Rest 호출
			if(String.valueOf(jsonResult.get("type")).equals("rest")){
				result = callWebApiRest(String.valueOf(jsonResult.get("url")), map);
			}
			//Soap 호출
			else if(String.valueOf(jsonResult.get("type")).equals("soap")){
				result = callWebApiSoap(String.valueOf(jsonResult.get("url")), map);
			}

		}catch(Exception e){
			e.printStackTrace();
		}

		return result;

	}

/**
 * 파일 첨부가 있을 경우 MultipartHttpServletRequest 형식으로 REST, SOAP 호출
 * @param request
 * @return
 */
@RequestMapping(value = "/selectUnitTestFile", method = RequestMethod.POST,
		produces = "application/text;charset=utf8")
	public @ResponseBody String selectUnitTest(MultipartHttpServletRequest request){

		String result = "";

		try{
			//Rest 호출
			if(String.valueOf(request.getParameter("type")).equals("rest")){
				result = ac.callWebApiRest(request);
			}
			//Soap 호출
			else if(String.valueOf(request.getParameter("type")).equals("soap")){
				result = ac.callWebApiSoap(request);
			}

		}catch(Exception e){
			e.printStackTrace();
		}

		return result;

	}

/**
 * 통합테스트 목록 조회
 * @param param
 * @return
 */
	@RequestMapping(value = "/unitTestInit", method = {RequestMethod.GET, RequestMethod.POST})
	public @ResponseBody String unitTestInit(@RequestBody HashMap<String, String> param){

		try {
```

```
                        String result = "";
                        //2. X509 Certificate Setting
                        HttpClient httpClient = ac.setAuthHttpClient();

                        //3. Host Url Setting
                        String listUri = param.get("url");
                        HttpGet httpGet = new HttpGet(listUri);
                        httpGet.addHeader("accept","application/json");

                        //4. Execute
                        HttpResponse response = httpClient.execute(httpGet);
                        HttpEntity httpEntity = response.getEntity();

                        result = EntityUtils.toString(httpEntity,"UTF-8");

                        return result;

                } catch (Exception e) {
                        e.printStackTrace();
                        return e.getMessage();
                }
        }


}
```

2.7. UI 구성예제

1) JSP 페이지에서 Spring Controller를 통한 WEB API 호출 예제

```
//검색 버튼 클릭 이벤트
$("#btnSearch").click(function(){

var paramUrl = encodeURI(wslimsUrl + "/webService/rest/selectUnityTestSenario", "UTF-8");
var unityTestSn = $("#unityTestSn option:selected").val();
var type = "rest";

var param = {"url" : paramUrl, "type" : type, "param" : {"unityTestSn" : unityTestSn}};

$.ajax({
        url : "/ApiClient/selectUnitTest",
        type : "POST",
        dataType : "json",
        data : JSON.stringify(param),
        contentType : "application/json; charset=utf-8",
        success:function(data){

                //테이블 생성
                ctrateTable(data);
        },
        error:function(request, status,error){
                console.log("code: " + request.status + "₩n" + "message: " +
                  request.responseText + "₩n" + "error: " + error);
        }
});

});
```

2) JSP 페이지에서 Spring Controller를 통한 WEB API 호출 예제

파일첨부 가 있는 API 와 일반 API를 파라미터로 구분한다.

```
/*REST, SOAP Service 실행*/
function startApiService(event){

        var param = "";
        var url = "";
        var type = "";

        //REST
        if(event == "rest"){
                url = wslimsUrl + "/webService/rest" + $("#txtRestUrl").val();
                type = "rest";
        }
        //SOAP
        else if(event == "soap"){
                url = $("#txtSoapUrl").val();
                type = "soap";
        }
```

```
        param = {"url" : url, "type" : type, "param" : $("#txtParameter").val()};

        /* 파일 정보가 있을때 multipart 형식으로 처리 */
        if(apiFileUpload == true){

                var input = new Array();

                $("#uploadForm").find("input").remove();

                //url 정보 input 생성
                input[0] = document.createElement("input");
                input[0].setAttribute("type", "hidden");
                input[0].setAttribute("name", "url");
                input[0].setAttribute("value", url);

                //type 정보 input 생성
                input[1] = document.createElement("input");
                input[1].setAttribute("type", "hidden");
                input[1].setAttribute("name", "type");
                input[1].setAttribute("value", type);

                //param 정보 input 생성
                input[2] = document.createElement("input");
                input[2].setAttribute("type", "hidden");
                input[2].setAttribute("name", "param");
                input[2].setAttribute("value", $("#txtParameter").val());

                //form 에 추가
                $("#uploadForm").append(input[0]);
                $("#uploadForm").append(input[1]);
                $("#uploadForm").append(input[2]);

                //form 정보를 formData 에 담기
                var formData = new FormData();

                formData.append("url", $("[name=url]").val());
                formData.append("type", $("[name=type]").val());
                formData.append("param", $("[name=param]").val());

                //업로드할 파일 개수 만큼  추가
                for(var i=0; i<$("[name=uploadFile]").length; i++){
                        formData.append("uploadFile", $("[name=uploadFile]")[i].files[0]);
                }

                $.ajax({
                        url : "/ApiClient/selectUnitTestFile",
                        type : "POST",
                        data : formData,
                        processData : false,
                        contentType : false,
                        success:function(data){
                                //조회 테이블 생성
                                ctrateTable(data);

                        },
                        error:function(request, status,error){
                                console.log("code: " + request.status + "₩n" + "message: " +
request.responseText + "₩n" + "error: " + error);
                        }
                });
        }
        else{
                $.ajax({
                        url : "/ApiClient/selectUnitTest",
                        type : "POST",
                        dataType : "json",
                        data : JSON.stringify(param),
                        contentType : "application/json; charset=utf-8",
                        success:function(data){
                                //조회 테이블 생성
                                ctrateTable(data);

                        },
                        error:function(request, status,error){
                                console.log("code: " + request.status + "₩n" + "message: " +
request.responseText + "₩n" + "error: " + error);
                        }
                });
        }
}
```

- 위 UI 예제는J SP 페이지에서 Spring Controller를 통한 WEB API 호출 하여 화면에 출력하는 예제이다.

2.8. 통합LIMS&검사기관 시스템 양방향 통신

- 통합LIMS 에서 이벤트 발생 시 검사기관으로 신호를 전달해주는 기능으로 통합LIMS에서 시료추적이력에 데이터가 등록 시 검사기관의 서비스를 비동기로 호출하여 검사기관에서 해당 검사데이터에 대한 추가 처리를 할 수 있도록 한다.

2.8.1. 사전준비사항

- 통합LIMS와 양방향 통신을 위해서는 검사기관 시스템에 URL로 호출가능한 형태의 서비스가 구축되어있어야 하며, 통합LIMS에 해당 검사기관의 웹서비스 정보가 등록되어있어야 가능하다.

2.8.2. 처리순서

1) 통합LIMS에서 데이터 처리 후 시료추적이력 등록 이벤트가 발생
2) 검사기관이 사전에 등록한 웹서비스 URL과 메소드를 비동기 방식으로 호출
   - 파라미터 : 통합LIMS변경구분코드 + 의뢰번호 + 시료순번 + 항목순번 + 검사기관 웹서비스 인증키
3) 검사기관 웹서비스에서 해당내용 수신 후 자동 처리

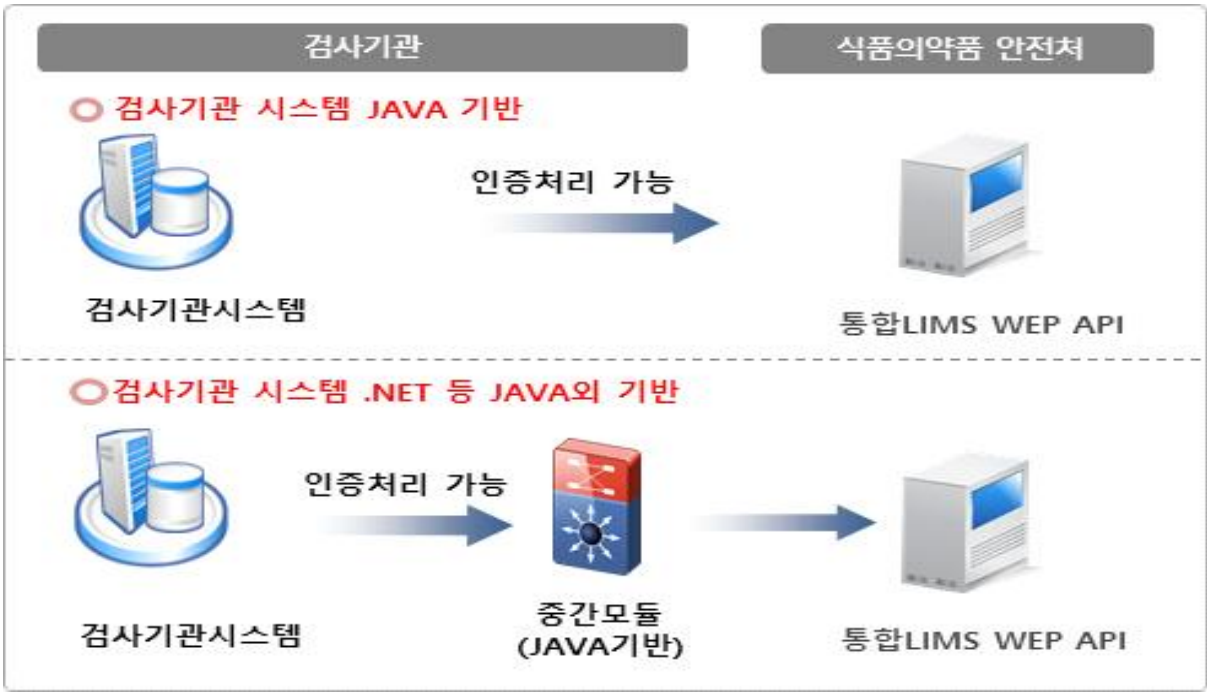
2.8.3. 통합LIMS 주요 변경 이벤트

변경구분	코드	변경구분	코드
접수(시험의뢰)	IM22000001	결재반려	IM2200012
접수 및 배정	IM22000003	성적서발급	IM2200013
시험일지저장	IM22000004	지연/철회/수정요청등록	IM2200014
시험결과입력	IM22000005	지연/철회/수정요청 처리	IM2200015
결과회수	IM22000007	담당자 및 처리기한변경	IM2200016
결과취합	IM22000008	취합자 및 결재자변경	IM2200017
결재회수	IM22000009	프로세스변경	IM2200018
결과취합반려	IM22000010	품목 및 항목변경	IM2200019
결재	IM22000011		

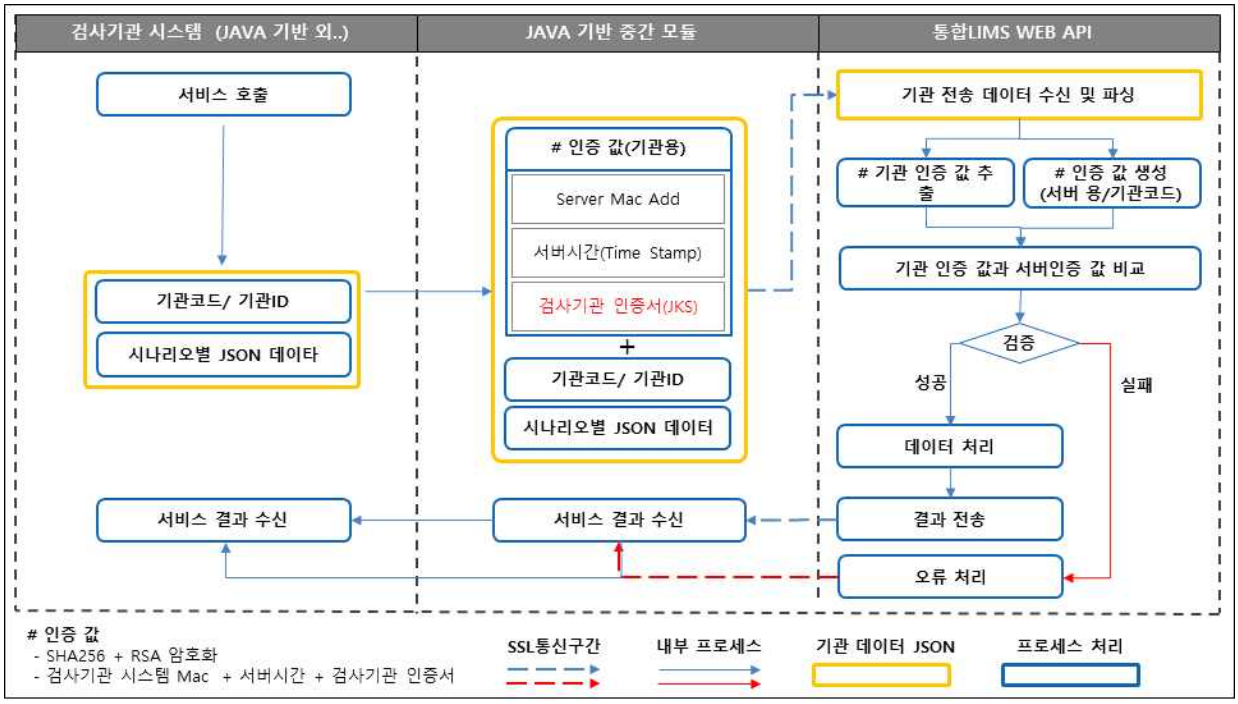
3. 중간 모듈(WEP SERVICE)

3.1. 개요

- 검사기관의 시스템이 .NET 등 JAVA 외의 시스템일 경우 JKS 인증서 인증처리가 불가능하여 기관의 검사데이터를 통합LIMS로 전송할 수 없기 때문에 중간모듈을 둠으로써 검사기관 시스템이 어떤 플랫폼으로 개발되어 있든 상관없이 기관의 검사데이터를 통합LIMS 로 전송할 수 있게 한다.



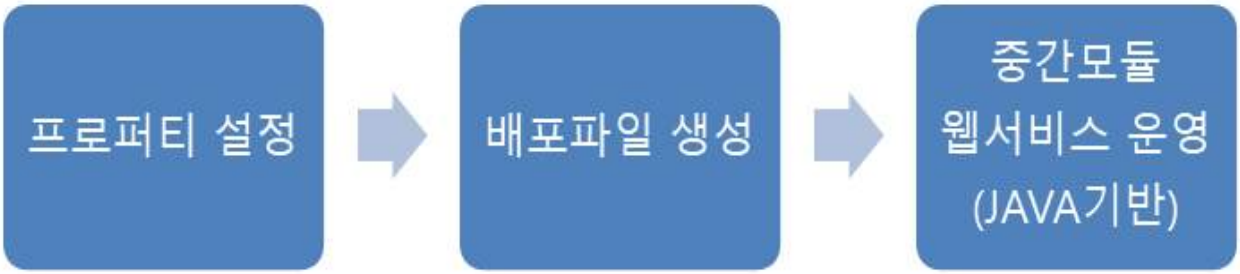
3.2. 중간모듈 프로세스



1) 검사기관 클라이언트에서 중간 모듈로 기관코드/기관ID, 시나리오별 데이터를 전송
2) 중간모듈에서 인증서정보를 탑재하여 통합LIMS WEB API 서비스호출
3) 통합LIMS WEB API 서버에서 서버인증서와 중간모듈의 인증서를 비교하여 유효성 확인
4) 검사기관의 서버 Mac Address 유효성 확인
5) 인증서 및 Mac Address 유효성 통과 시 요청 웹서비스 호출
6) 요청 웹서비스의 입력 파라미터값을 검증 후 Validation Check
7) 처리결과 리턴

3.3. 중간 모듈 가이드

3.3.1. 서비스 환경설정



1) \검사기관개발가이드_v1.1\Sample_중간모듈\web Service(중간모듈) 경로의 workspace 세팅
2) 중간 모듈 프로퍼티의 인증서 파일경로, 인증서 암호, 인증서 명 등을 수정
3) 중간모듈을 운영할 웹서비스에 맞춰서 배포파일 생성
4) JAVA 기반의 웹 서버에 배포파일을 배치 후 실행

1) 중간모듈 사용 환경

개발 언어	JAVA
자바 버전	jdk 1.7 이상
프레임워크	spring(maven)
	bootstrap
웹서비스	apache cxf 3.1.0
서블릿 컨테이너	tomcat 7 이상 혹은 JAVA 기반 웹서버
인증서타입	JKS(Java Key Store)

2) 중간 모듈 properties 설정

```
//인증서 파일 경로
keyStore.keyStoreLoc=C:/CA/O000170.jks
//인증서 암호
KeyStore.keyPassWord=mfds2015
//인증서 명칭
keyStore.keyAlias=O000170
//파일 다운로드 경로
fileDownloadPath=C:/mfdshome/files/client
//통합 LIMS URL
wslimsUrl=https://wslims.mfds.go.kr
```

3.3.2. 서비스 운영방법

본 문서에서는 visual studio + C# + Sample 소스(검사기관클라이언트_Sample)를 기본으로 설명한다.

1) 기관코드/기관ID, 시나리오별 DATA 설정

※ 서비스 별 필요한 입력 Parameter정보는 서비스 별 API 가이드 참조

```
$('#btnSet').click(function () {
    var param = "";
    //rest, soap urlbrsiol 다름
    if ($('#txtTran').val() == "soap") {
        param = {
            "url": "/selectListCmmnCode"            // 메뉴 URL
            "type": "soap",                         // 전송방식
            "param": {"mfdsLimsId": "apitest01", "psitnInsttCode": "O000170", "classCode": "IM36"}
                     // mfdsLimsId 사용자ID , psitnInsttCode 사용자기관코드 classCdoe 분류코드
        };
    } else {
        param = {
            "url": "https://wslims.mfds.go.kr/webService/rest" + "/selectListCmmnCode",
            "type": "rest",
            "param": {"mfdsLimsId": "apitest01", "psitnInsttCode": "O000170", "classCode": "IM36"}
                     // mfdsLimsId 사용자ID , psitnInsttCode 사용자기관코드 classCdoe 분류코드
```

```
        };

    }
});
```

2) 수신URL 설정, 데이터 전송

```
$('#btnCon').click(function () {

    // 파일정보 유무 체크
    var chkFile = ($('#uploadFile').val())
    if (!chkFile) {
        $.ajax({
            url: "http://localhost:8080/selectUnitTest", //중간모듈 URL
            type: "POST",
            //dataType: "json",
            data: JSON.stringify(param),
            contentType: "application/json; charset=utf-8",
            success: function (data) {
                console.log(data);
                //조회 테이블 생성
                ctrateTable(data);
            },
            error: function (request, status, error) {
                console.log("code: " + request.status + "₩n" + "message: " +
request.responseText + "₩n" + "error: " + error);
            }
        });
    } else {
        var input = new Array();

        $("#uploadForm").find("input").remove();

        //url 정보 input 생성
        input[0] = document.createElement("input");
        input[0].setAttribute("type", "hidden");
        input[0].setAttribute("name", "url");
        input[0].setAttribute("value", param.url);

        //type 정보 input 생성
        input[1] = document.createElement("input");
        input[1].setAttribute("type", "hidden");
        input[1].setAttribute("name", "type");
        input[1].setAttribute("value", param.type);

        //param 정보 input 생성
        input[2] = document.createElement("input");
        input[2].setAttribute("type", "hidden");
        input[2].setAttribute("name", "param");
        input[2].setAttribute("value", param.param);

        //form 에 추가
        $("#uploadForm").append(input[0]);
        $("#uploadForm").append(input[1]);
        $("#uploadForm").append(input[2]);

        //form 정보를 formData 에 담기
        var formData = new FormData();

        formData.append("url", $("[name=url]").val());
        formData.append("type", $("[name=type]").val());
        formData.append("param", $("[name=param]").val());

        //업로드할 파일 개수 만큼  추가
        for (var i = 0; i < $("[name=uploadFile]").length; i++) {
            formData.append("uploadFile", $("[name=uploadFile]")[i].files[0]);
        }

        $.ajax({
            url: "http://localhost:8080/selectUnitTest", //중간모듈 URL
            type: "POST",
            data: formData,
            processData: false,
            contentType: false,
            success: function (data) {
                //조회 테이블 생성
                ctrateTable(data);
                console.log(data);
            },
            error: function (request, status, error) {
```

```
                console.log("code: " + request.status + "₩n" + "message: " +
request.responseText + "₩n" + "error: " + error);
            }
        });
    }

});
```

3) 결과 출력

```
function ctrateTable(result) {
        var data= decodeURIComponent(decodeURI(result).replace(/₩+/g, " "))
        var str = "";

        //생성된 테이블 삭제
        $("#resultData table").remove();
        $("#resultData div").remove();

        //panel 크기 조정
        $("#resultBody").css("height", "700px");
        $("#resultBody").css("width", "100%");
        //테이블이 들어갈 공간 크기 조정
        $("#resultData").css("height", "650px");

        //Vo Header 가 있을 경우 테이블 생성
        if (data.header != null) {
            var header = data.header; //헤더  저장
            var resultData = data.resultData; // 내용 저장
            var cnt = 0; //No. 생성


            str += "<table id='resultTable' class='table table-bordered' style='table-layout:fixed;'>";

            str += "<colgroup>";
            str += "<col style='width:50px;'>";
            $.each(header, function (index, item) {
                str += "<col style='width:" + (item.length * 50) + "px;'>";

            });
            str += "</colgroup>";

            str += "<thead class='thead-dark'>";
            str += "<tr>";
            str += "<th>순번</th>";


            //header 생성
            $.each(header, function (index, item) {
                str += "<th>" + item + "</th>";
            });

            str += "</tr>";
            str += "</thead>";

            //td 내용 생성
            $.each(resultData, function (rIndex, rItem) {
                cnt++;
                str += "<tr>";
                str += "<td>" + cnt + "</td>";

                $.each(header, function (hIndex, hItem) {
                    str += "<td>";

                    if (!rItem[hIndex]) {
                        str += "";
                    }
                    else {
                        str += rItem[hIndex];
                    }

                    str += "</td>";
                });

                str += "</tr>";
            });

            str += "</table>";
```

```
        }
        //Vo Header 가 없을 경우
        else {

            //결과 초기화
            $("#resultData").text("");

            //결과값 append
            str += "<div class='row'>";
            str += data;
            str += "</div>";

            //파일 다운로드 url 비교 하여 파일 다운로드 파일 생성
            var resultUrl = $("#txtRestUrl").val();
            var soapUrl = $("#txtSoapUrl").val();

            str += "<br/>";

            if (resultUrl == "/selectAtchmnflDwld" || soapUrl == "/selectAtchmnflDwldSoap" ||
                resultUrl == "/selectGrdcrtPrevew" || soapUrl == "/selectGrdcrtPrevewSoap" ||
                resultUrl == "/selectGrdcrtDmOutpt" || soapUrl == "/selectGrdcrtDmOutptSoap" ||
                resultUrl == "/selectGrdcrtPdfOutpt" || soapUrl == "/selectGrdcrtPdfOutptSoap" ||
                resultUrl == "/selectGrdcrtOutpt" || soapUrl == "/selectGrdcrtOutptSoap" ||
                resultUrl == "/selectReqestRceptRdmsPdf" || soapUrl ==
"/selectReqestRceptRdmsPdfSoap") {

                str += "<div class='row'>";
                str += "<button type='button' class='btn btn-primary' onClick='onDownLoad(" +
JSON.stringify(data) + ")'>파일 다운로드</button>";
                str += "</div>";

            }


        }

        setTimeout(function () {
            $("#resultData").append(str);
        }, 200);

    }

    /* 파일 다운로드 */
    function onDownLoad(resultData) {
        var fileName = resultData["resultData"][0]["fileName"];

        $(location).attr("href", "/ApiClient/unitTestFileDownLoad?fileName=" + fileName + "");
    }
```